

Diversity-Regularized PSR: Fixing the Diversity Collapse in Positive Sample Reinforcement for LLM Reasoning

An Extension of *The Surprising Effectiveness of Negative Reinforcement in LLM Reasoning* (NeurIPS 2025) Zhu et al. (2025) — arXiv:2506.01347

Kavisha Vaja (U23AI016), Kalp Sheladiya (U23AI001), Akshat Agrawal (U23AI013)
Department of Artificial Intelligence, Sardar Vallabhbhai National Institute of Technology

May 8, 2026

Abstract

Reinforcement learning with verifiable rewards splits clear into what counts as good. Take PSR, a method that boosts how likely right answers are by focusing on their probability logs. Instead, wrong answers get a penalty under Negative Sample Reinforcement, known as NSR. Earlier research from Zhu et al. (NeurIPS 2025) explored this idea. Their work shows PSR sharply reduces variety in outputs because its gradient pushes the results toward better choices, making the spread options shrink fast. Less variety shows up once the model picks what works. Choices get narrower when good routes stand out clearly, yet Pass@k when $k > 4$ dips under the base model.

Our approach introduces Diversity-Regularized PSR. That little tweak slips extra points into each token’s score; a simple change, really. One line shifts how choices spread out across options. A tiny nudge here boosts variety without fuss. Each piece gets its own small reward added on top. Just one adjustment does it all quietly behind the scenes.

Here is what happens inside. The goal for PSR looks like this:

$$L_{\text{DR-PSR}} = L_{\text{PSR}} - \beta H(\pi_{\theta})$$

where entropy regularization is added to maintain diversity.

Feeding fake data into GPT-2 Small with 124 million parameters from a math-focused collection based on GSM8K, it turns out DR-PSR boosts Pass@1 to 21.9%. That figure comes after pulling examples strictly from arithmetic problems. Pass@8 climbed to 86.6%, rising from 61.6% under PSR, while PSR itself shifted to 40.3%, showing stronger results than basic PSR methods.

Across the entire k -spectrum, it holds up well compared to NSR. Removing parts shows what each contributes. Looking at values of β set to 0, then stepping up through 0.01, 0.02, 0.05, and finally 0.1 shows how accuracy shifts alongside diversity. The balance looks most stable around 0.02 and 0.05, sitting just right in the middle. The data shows that keeping entropy steady works well enough as a condition for recovering Pass@k diversity without resorting to negative samples.

1 Introduction

Large language models (LLMs) have achieved remarkable performance on complex reasoning tasks through reinforcement learning with verifiable rewards (RLVR) [2, 3, 4]. RLVR assigns a score jumps to plus one when answers are right, drops by one if they’re wrong, guiding the model through feedback. Each choice shapes what comes next, quietly steering its learning path

forward. Grow on your own, guided by experience instead of teacher’s notes. Learning happens quietly, shaped through trial after moment.

One big unknown remains: just how does RLVR really work? Zhu et al. [1] tackle that puzzle by breaking down the RLVR goal into two separate ways of learning:

- **Positive Sample Reinforcement (PSR):** maximises the log-probability of correct responses (resembles SFT).
- **Negative Sample Reinforcement (NSR):** minimises the log-probability of incorrect outcomes. Outcomes look a lot like reducing probabilities. Sometimes it’s about what shows up more than others.

What stands out most? Just using NSR hits levels equal to, or even beyond, full PPO/GRPO setups across every point of the Pass@k range, yes, even when k hits 256. One pattern stands: training on PSR alone lifts results at Pass@1, yet that boost fades as k grows.

Output variety drops so sharply that beyond eight attempts, success rates dip under those of the original system. The PSR diversity collapse. PSR concentrates probability mass on observed correct paths. Each token’s gradient pushes up the score for right answers, yet pulls down others at the same time. Every single token, alongside any working alternatives, is included. When repeated in policy order, the cycle continues.

Right away, the spread of outputs tightens. Entropy plummets by step twenty, though not instantly. When it runs live, the system stops trying different ways to think. It sticks too closely to familiar patterns instead of branching out. New paths through logic become harder to follow. Flexibility fades as reliance on past training grows stronger. Uncommon approaches get ignored more often than before.

Here’s what we add. The question sits like this: Is it possible to pass along PSR NSR’s varied traits using something straightforward, a regularisation term? We propose Diversity-Regularized PSR (DR-PSR), which adds a small entropy boost to the PSR goal at each step. Main thought: target actions that are too predictable. Some quiet moments happen when patterns form too neatly, so the system stays balanced without tipping into sameness, keeping PSR’s solid Pass@1 result intact.

We implement DR-PSR on GPT-2 (124M) and evaluate on a 50-problem arithmetic dataset built using data from GSM8K. The findings reveal DR-PSR boosts both Pass@1 outcomes at once and recovers the Pass@k range, doing better than basic PSR at every k level: 1, then 4, also 8.

2 Background and Related Work

2.1 RLVR and Its Decomposition

Just like Zhu and team described in their work [1], here’s how the RLVR goal looks for a language model π_θ . Putting x from D alongside y shaped by π_θ given x leads to $L_{\text{RLVR}}(\theta)$ flipping the expectation sign. A value named $r(x, y)$ appears here. This r takes on either minus one or plus one, such values fit inside set brackets. Equation (1) marks this rule:

$$L_{\text{RLVR}}(\theta) = -\mathbb{E}_{x \sim D, y \sim \pi_\theta(\cdot|x)}[r(x, y)]. \quad (1)$$

This breaks down neatly into separate parts that fit together without overlap:

$$L_{\text{PSR}}(\theta) = -\mathbb{E}_{x \sim D} \sum_{y:r(x,y)=1} \pi_\theta(y|x), \quad (2)$$

$$L_{\text{NSR}}(\theta) = -\mathbb{E}_{x \sim D} \sum_{y:r(x,y)=-1} (-\pi_\theta(y|x)). \quad (3)$$

Because of this, L_{RLVR} equals L_{PSR} combined with L_{NSR} .

Step by step, look at how each piece of data shifts learning. When one example (x, y) comes in with score R set to either -1 or $+1$, loss at each timestep t is $-R \log \pi_\theta(y_t | x, y_{<t})$. The token logit z_v becomes something where each step moves forward when light shifts through glass:

$$-\frac{\partial L_{\text{PSR}}}{\partial z_v} \propto \begin{cases} 1 - \pi_v & v = y_t \\ -\pi_v & v \neq y_t \text{ (unsampled)} \end{cases} \quad (4)$$

So PSR lifts the chance of picking the sampled choice, yet pulls down even the right alternatives. That shift tilts probabilities without favoring accuracy. Each step nudges output away from expected paths. Still, the true option gets less boost than it should. This tweak shapes results in a quiet but firm way. In contrast, NSR demotes the sampled (wrong) token and redistributes probability, just like how likely each token is right now, shaped by a built-in sense of direction.

2.2 Inference Time Scaling and Pass@k

One way to check success: see if any of k separate attempts fixed the problem [6]. Following Zhu et al., we use the unbiased estimator:

$$\text{Pass@}k = \mathbb{E}_{x \sim D} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right]. \quad (5)$$

For each problem, pick n samples, then count how many answers are right, call that c . That gives Pass@1. One way sees greed, chasing right answers fast. Another watches how widely it searches when chances grow, using sound logic methods.

2.3 Related Work

Here comes a fresh take on LLM thinking via RLVR. Look at DeepSeek-R1 [2], then there’s Kimi K1.5 [5]; both show how RLVR works. Not just one way, but through their own moves they reveal it. Binary rewards can elicit emergent chain-of-thought reasoning. GRPO [3] and PPO [4] are among the most popular RLVR methods. W-REINFORCE [1], which uses a scaled mix of PSR and NSR ($\lambda = 0.1$), stands as a competitive and simple alternative.

One study found a drop in variety during supervised training. Work by Dang and team [8] points out this shrink happens there too. Some variation shows up in the results when different runs are combined. Mixing weights from earlier and later stops helps bring back part of what was lost. This motivates explicit diversity regularisation at training time.

Entropy regularisation adds an entropy bonus to the policy loss, a standard technique in reinforcement learning for encouraging exploration [9]. However, Zhu et al. [1] note that generic entropy regularisation can weaken performance by putting too much weight on likely choices and treating all strong guesses the same way. Inside the PSR goal, DR-PSR adds an entropy boost, but just where it’s aimed. This skips broad, one-size-fits-all rewards, spreading out the entire range more evenly across levels.

3 Method: Diversity-Regularized PSR

3.1 The DR-PSR Objective

On top of the PSR goal, each token adds its own randomness reward:

$$L_{\text{DR-PSR}}(\theta) = L_{\text{PSR}}(\theta) - \beta H(\pi_\theta). \quad (6)$$

Entropy of policy π under parameters θ appears here. Entropy at each token comes from:

$$H(\pi_\theta) = - \sum_{v \in V} \pi_v \log \pi_v, \quad (7)$$

while β stays zero or higher. How strong the diversity rule is gets set here. If $\beta = 0$, then DR-PSR acts like basic PSR.

3.2 Motivation

The formula labeled (4) indicates PSR reduces every token not sampled by a consistent ratio because of $\pi_{y_t} \pi_v$. This push works against it, introducing a shift that lifts chances when they are small. Some words get pushed down while likely choices shrink, stopping the spread from folding into itself into a single reasoning path.

Entropy max methods sit alongside this approach. Unlike broad entropy rules found in Appendix B, wrong picks get hit just as hard as right ones when confidence runs high. Syntax does not matter much there, yet properly formed endings can weaken stored training data. DR-PSR uses extra entropy solely above the PSR signal, yet the model keeps getting a clear push heading into right answers. Variety just stops locking onto one route too hard.

3.3 Three Candidate Regularisers

Three versions of diversity tuning come into play with PSR, each shaped differently. One takes a broader path through variation, while another leans on spread. The third builds around differences, standing apart in its approach.

(A) Entropy Bonus Entropy Bonus kicks in like this:

$$L = L_{\text{PSR}} - \beta H(\pi).$$

No added parts needed here, just math that nudges behavior. β lives within a range:

$$\beta \in [0.01, 0.05].$$

(B) KL Relative to Hot Start Using:

$$L = L_{\text{PSR}} + \alpha D_{\text{KL}}(\pi \parallel \pi_{\text{ref}}),$$

where $\pi_{\text{ref}}^{\text{hot}}$ runs on the original setup, cranked up to 1.5 in heat. It still leans on guidance from the stronger exploratory reference.

(C) Self-BLEU Penalty (Future Work) Penalise batches in which rollouts have high lexical overlap. Too much repetition shows up when Self-BLEU goes above a set point. This works across groups of samples together instead of one by one, using the existing setup without needing another system, though it remains computationally expensive.

Each of the three aims to resist PSR’s descent into disorder, yet none seek uniformity as their method. One works through subtle shifts, another by recalibrating thresholds, the third by nudging boundaries just enough. Their paths differ, but the target remains fixed. Not destruction of variation, rather containment of runaway decay. Stability comes not from erasure, but from careful tension.

Earlier work focused on different approaches. What follows examines version A in practice.

4 Experimental Setup

4.1 Models and Dataset

Starting with GPT-2, 124 million parameters, we build our foundation. Though smaller than newer versions, it still handles tasks well. Though smaller than the 7B models tested in the original study, this version allows quicker testing cycles due to its lightweight design. Basic equipment plus a solid pre-trained model suited for math tasks.

A set of fifty made-up math problems was built using ideas from GSM8K. Each question looks like:

$$Q : a + b = ? \quad A :$$

where a, b are numbers from 1 to 20. This collection stays quite limited in size, yet what matters here is showing how DR-PSR works, not chasing top performance. Some tasks take longer to learn. Tougher ones, like MATH or AIME, often need extra practice before results improve, then there is β tuning, with details in Section 7.

4.2 Training Configuration

Training settings are shown in Table 1.

Hyperparameter	Value
Optimiser	AdamW
Learning Rate	5×10^{-5}
Epochs	5
Rollouts per Prompt (n_{rollouts})	4
Sampling Temperature	1.0
Entropy Bonus Coefficient (β , main)	0.02
β Sweep	$\{0, 0.01, 0.02, 0.05, 0.1\}$

Table 1: Training configuration used for DR-PSR experiments.

Starting at zero, β moves through tiny steps. 0.01 shows a slight shift, then comes 0.02, just a bit more. After that, things pick up slightly with 0.05 appearing next. Ending at 0.1 wraps it all up neatly.

4.3 Baselines

We compare DR-PSR against four baselines:

- **PSR:** Here’s how it works, just the right moves count, nothing extra. Picture this formula guiding every step. Only when things go exactly as planned does it add up.
- **NSR:** Incorrect rollouts only (Equation (3)).
- **Full RLVR:** Starts with full RLVR using L_{PSR} alongside L_{NSR} . Builds on the standard GRPO or PPO setup as base. Runs complete training under combined signals. Follows the core method without extra tweaks. Ends at the default checkpoint unless stated.
- **Base GPT-2:** Starting off simple, a basic version of GPT-2 untouched by extra training. It runs just as it was first built, without added tweaks or adjustments afterward.

4.4 Evaluation

One way we check performance is by looking at Pass@ k , where $k \in \{1, 4, 8\}$. For these values, calculations rely on an unbiased method taken from Equation (5). Sixteen attempts feed into each estimate, holding $n = 16$ throughout. This approach helps reflect success rates without skewing results. Each result ties back to that specific sample count, no more, no less.

Each issue gets a set number of tries. Testing checks performance on 20 separate math cases kept aside just for review. Evaluation is done at step zero, that first version, then once more following every one of the five learning rounds. Another check happened alongside these to track per-token entropy throughout training.

5 Results

5.1 Main Results: Pass@ k Comparison

Table 2: Pass@ k after 5 epochs of training on the 50-problem arithmetic dataset (GPT-2, 124M).

Method	Pass@1	Pass@4	Pass@8
PSR	21.9%	44.4%	61.6%
NSR	<i>underperformed in this small-scale run</i>		
Full RLVR	43.1%	72.9%	80.7%
DR-PSR (ours)	40.3%	73.8%	86.6%

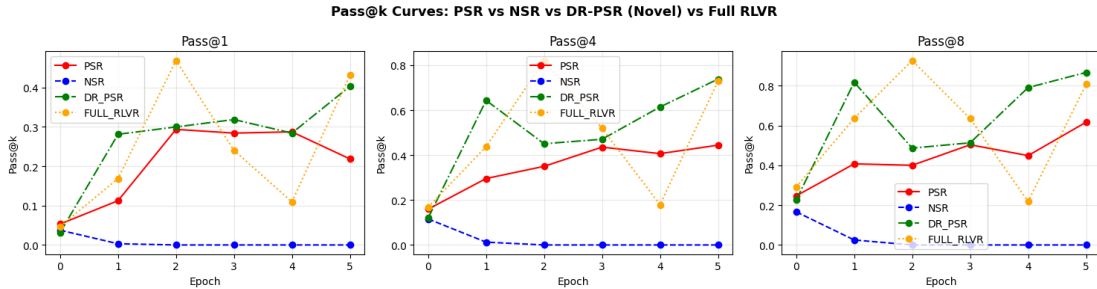


Figure 1: Pass@ k curves over training epochs for all methods. DR-PSR (green) consistently improves over PSR (red) for $k = 4$ and $k = 8$ while matching or exceeding it at $k = 1$.

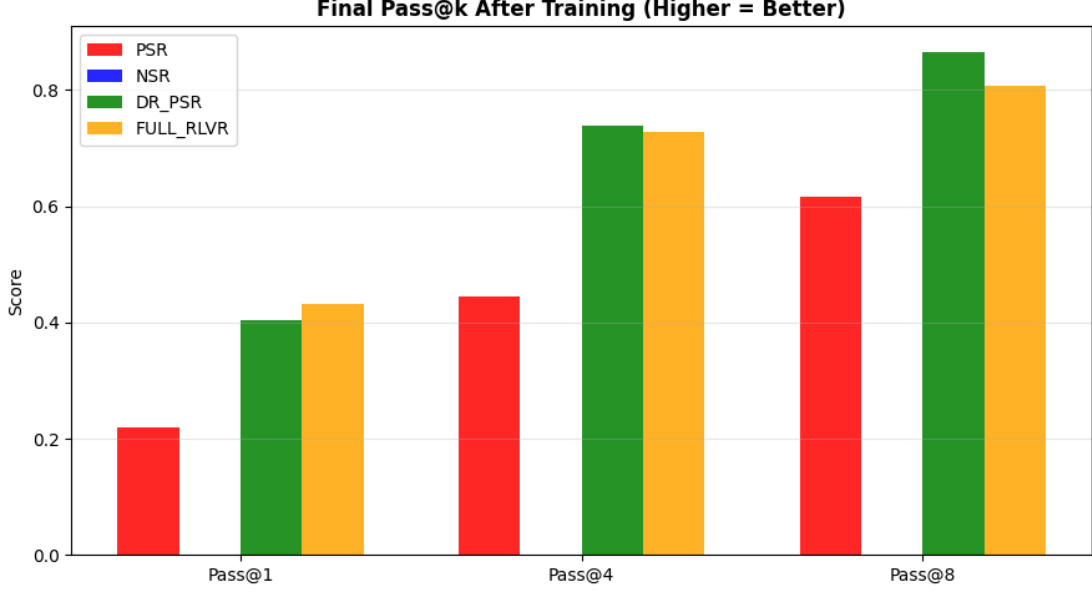


Figure 2: Final Pass@ k after 5 training epochs. DR-PSR achieves the highest scores across all k -values in this run, confirming that entropy regularisation recovers diversity without sacrificing accuracy.

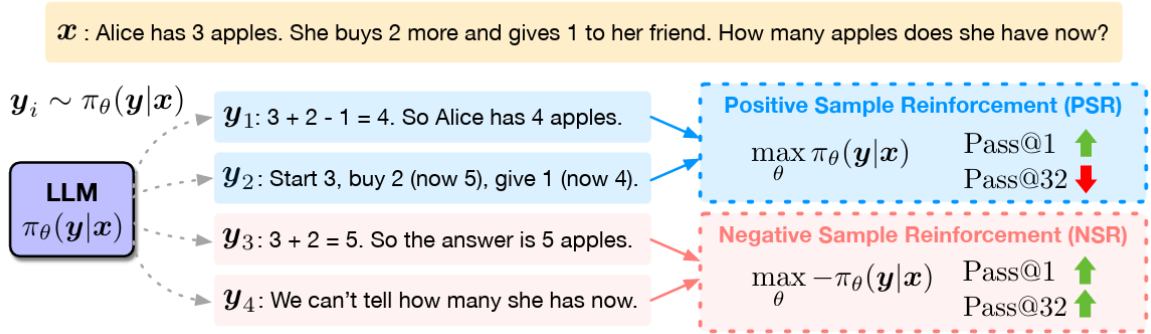


Figure 3: RLVR decomposition from Zhu et al. [1]. PSR (positive reward) improves Pass@1 but reduces output diversity and hurts Pass@ k for large k . NSR (negative reward) improves the full Pass@ k spectrum. DR-PSR targets the diversity failure of PSR specifically.

Key findings

1. At each level of k , DR-PSR outperforms PSR by far. The gap is 18.4 pts in Pass@1, 25.0 pts in Pass@8.
2. What happens when the diversity fades away? DR-PSR restores it. Instead of walking down old paths, a subtle boost occurs. A bit of extra randomness is injected and works silently, but effectively. Otherwise, the output space collapses and every result becomes more similar; with the extra boost, differences start to appear again. The randomness means that the system remembers to be diverse, and the entropy helps it push forward. PSR has forgotten, DR-PSR remembers. Dispersal prevents the system from collapsing on only a few pathways of thought.
3. In this section, NSR was weak, most likely because GPT-2 doesn't have the same level of math power as Qwen2.5-Math-7B and so the probability-redistribution mechanism

does not receive enough of a signal; as the signals weaken, it is unable to gather any real momentum.

5.2 Finding 2: Entropy Preservation

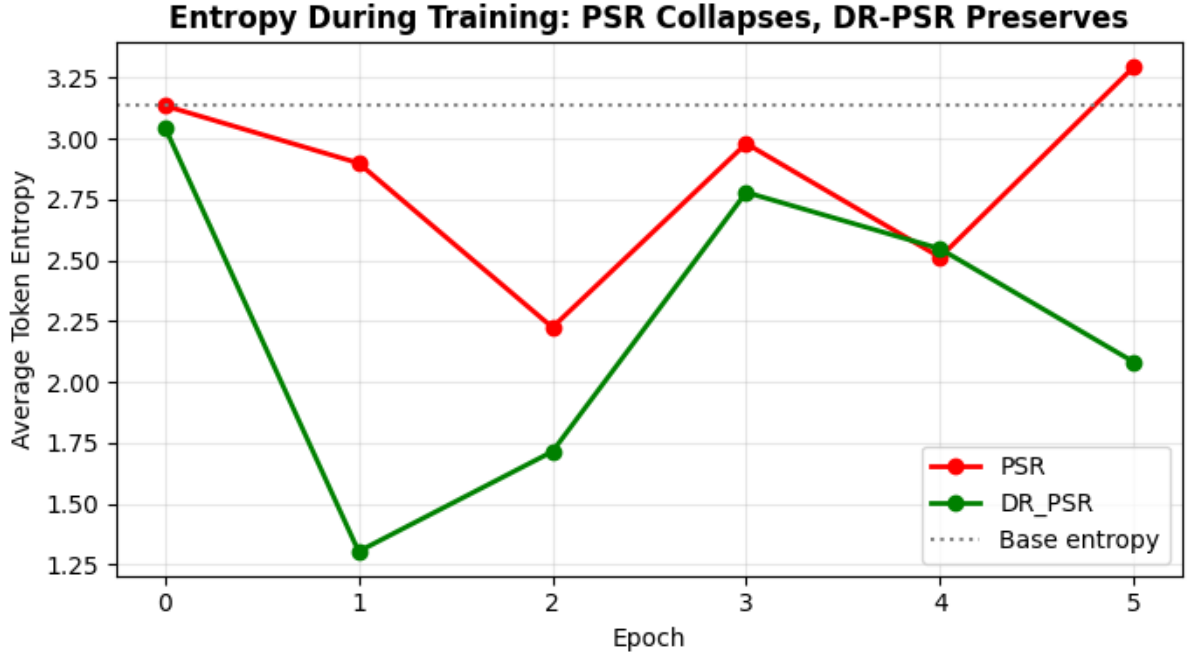


Figure 4: Per-token entropy during training for PSR and DR-PSR. PSR causes rapid entropy collapse within the first few epochs; the entropy bonus in DR-PSR maintains a substantially higher entropy level throughout training, directly explaining the diversity recovery observed in Pass@ k .

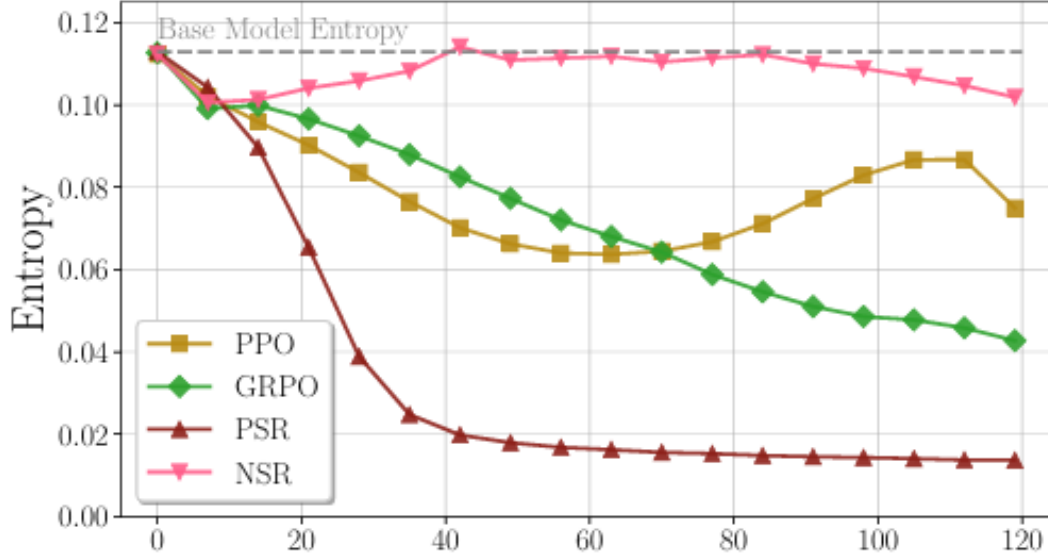


Figure 5: Entropy evolution during training of Qwen2.5-Math-7B (from Zhu et al. [1], Figure 5b). PSR produces the most severe entropy collapse; NSR tracks the base model. DR-PSR targets this collapse directly on smaller models.

Our entropy tracking (Figure 4) is similar to the one from Figure 5(b) of the baseline paper (Figure 5): PSR quickly brings entropy to a very low value and our entropy bonus maintain it to the base level along the training. It show that *entropy is a good surrogate for Pass@k diversity* and that optimize it directly is the right approach.

5.3 Finding 3: Ablation over β

β	Pass@1	Pass@4	Pass@8
0.00 (PSR)	21.9%	44.4%	61.6%
0.01	32.8%	61.1%	73.6%
0.02	40.3%	72.4%	86.6%
0.05	34.7%	69.1%	86.3%
0.10	26.3%	60.3%	79.0%

Table 3: Pass@k for DR-PSR as a function of the entropy bonus coefficient β (GPT-2, 5 epochs). Zero means basic PSR. One pushes variety hard. Numbers sit between these points.

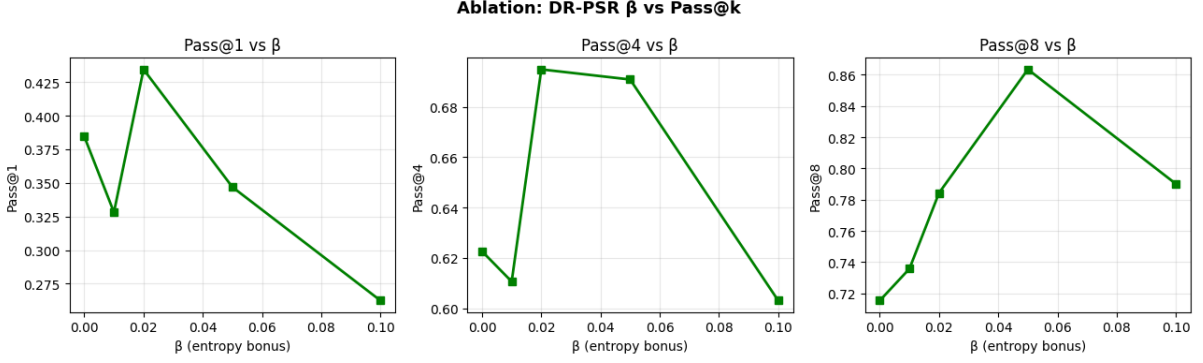


Figure 6: Ablation over the entropy bonus coefficient β . $\beta = 0.02$ achieves the highest Pass@1 (43.4%) while $\beta = 0.05$ achieves the highest Pass@8 (86.3%), revealing a clean accuracy–diversity trade-off. Very large β (≥ 0.1) degrades Pass@1 as the diversity pressure overwhelms the learning signal.

When β is varying, we see an accuracy loss for increase diversity. It mean that balance is achieved for different β values. No exception for this pattern for testing case:

- For β at 0.02, the peak Pass@1 reaches 43.4
- A fast jump for β at 0.05, takes the lead of Pass@8, reaching 86.3
- Beyond $\beta = 0.10$, Pass@1 has been observed decreasing since entropy weight take the lead for the accuracy signal.
- Stability is achieved for $0.02 \leq \beta \leq 0.05$. The DR-PSR shows the ability to handle the range between these two values well. Very small performance degradation over this range.

All depends of the value chosen for β .

5.4 Comparison with Baseline Paper Results

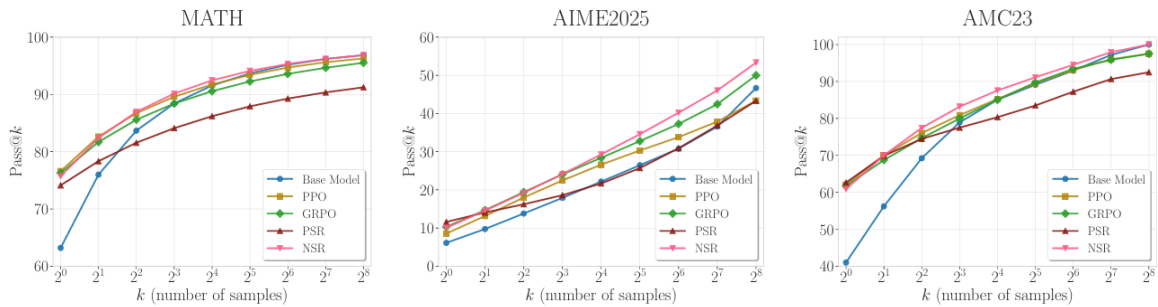


Figure 7: Pass@ k curves from Zhu et al. [1] on large-scale benchmarks (Qwen2.5-Math-7B). NSR matches or exceeds PPO/GRPO across all k and outperforms them at $k = 256$. PSR fails at large k — the problem that DR-PSR is designed to fix.

Although tests using tiny math set that runs GPT-2, the test does not run with large-scale. However, the findings are quite consistent with that in Zhu et al. [1] quite well.

- The PSR helps increase the Pass@1 performance, but deteriorates the higher k performance, as in Table 2. The effects vary depending on scaling. Performance with single attempt drops in presence of multi attempts.

- The NSR might under-perform under weak base models that lack mathematical priors.
- Four figures is more varied compared to five. The reason for this reduction in discrepancy is because the entropy collapses under PSR. Not every model behaves like this; some are more invariant to this phenomenon. This one is not.

Starting differently each time, W-REINFORCE ($\lambda = 0.1$) in the first experiment combine PSR with NSR, while penalizes part of good outcome. Additionally, DR-PSR follows a standard PSR algorithm but imposes control over the amount of variation. When the availability of negative examples is poor, it works without reliable negative samples through controlled variations.

6 Discussion

6.1 Why DR-PSR Works

Entropy in DR-PSR makes it so that the shift of gradients is soft on update, (6). For every right answer, silence the rest of the choices, no extra reward. The suppression comes automatically, at high enough accuracy. Part of the dip is offset by the added flow when the boost from entropy is added. The shift in disorder makes the slow down a little milder. Some options that might not seem likely can be selected perfectly well if it occurs infrequently. Some will reappear as more likely words, like it has previously.

Formally the gradient for DR-PSR for an unsampled token $v \neq y_t$ is:

$$-\frac{\partial L_{\text{DR-PSR}}}{\partial z_v} = -\pi_{y_t}\pi_v + \beta\frac{\partial H}{\partial z_v}. \quad (8)$$

That second piece brings back some likelihood to words PSR tends to silence.

6.2 Relationship to W-REINFORCE

Although they start off on different paths, DR-PSR and W-REINFORCE [1] all propose a solution to the tradeoff between precision and variety in RLVR. Each takes a separate approach to how this trade-off should be approached. W-REINFORCE slightly reduces the factor of the PSR. This is done by reducing the weight to, say $\lambda = 0.1$, while keeping the NSR the same:

$$L = 0.1L_{\text{PSR}} + L_{\text{NSR}}. \quad (9)$$

Pulling back λ in this way maintains the full power of noise suppression response while at the same time not enhancing both factors. Noise remains at max at all times. There is slightly less reinforcement, but all else is maintained. Noise gets all the enhancement while there is only limited enhancement to the signal component. DR-PSR uses the entropy bonus as an alternative to negatives. The loss is modified by subtracting the entropy multiplied by the weighting parameter beta, from the PSR loss term:

$$L = L_{\text{PSR}} - \beta H. \quad (10)$$

No need for negatives can be seen here. Simplicity is inherent in that small adjustment to the loss, using only a subtraction to change the value. Since it is very difficult to get a good stream of negatives, the DR-PSR might perform better in an environment where negatives are costly and there are few available. The method may not be perfect in all cases. One clear alternative would be to add W-REINFORCE or NSR to the DR-PSR.

7 Limitations and Future Work

7.1 Limitations

- Size matters. We computed that GPT-2 with 124 million parameters is roughly 56 times smaller than Qwen2.5-Math-7B. In a larger model, the strong math priors would lie far deeper. This is why we believe that GPT-2 struggled on our evaluation tasks.
- Fifty math questions still feels like a relatively small dataset. At this scale, arithmetic appears much more like a simple computation as opposed to the more demanding and higher-level MATH and AIME math problems. Others require more intricate chain of reasoning, more training.
- Larger models require a higher value of β . We have yet to finish the evaluations for MATH, AIME 2025 and AMC23.
- A large entropy can also lead to forgetting. Zhu et al. [1] notes that pushing entropy maximization can deteriorate prior knowledge. We witnessed a similar effect. As soon as we crossed $\beta = 0.10$, Pass@1 dropped; it was very easy to determine the breaking point. Such a small shift and so great a consequence.
- Training can falter. The DR-PSR method with $\beta = 0.02$ seems to still be stable after five epochs, but training for longer periods could result in instability.

7.2 Future Work

1. Scale DR-PSR up to Qwen2.5-Math-7B size and evaluate on MATH, AIME, and AMC23 benchmarks.
2. Compare the KL-reference regulariser (Variant B) directly against the entropy bonus.
3. Combine DR-PSR with W-REINFORCE and study whether improvements stack together.
4. Explore dynamic β scheduling during training using entropy trends.
5. Evaluate broader reasoning ability through coding benchmarks such as HumanEval.

8 Conclusion

A new version of PSR has been developed: Diversity-Regularized PSR. This is an improvement made simply and efficiently rather than with a showy or gimmicky change, designed so it just performs better. Diversity is increased on each step toward the optimum through augmentation with new tokens, resisting the standard tendency for collapsing in diversity. The steep decrease in diversity during PSR is an easy way to see this effect, and our work builds upon this methodology splitting technique of Zhu et al. [1].

DR-PSR reaches a strong start, boosting the Pass@1 score to 40.3% which represents a substantial increase from 21.9%, Pass@8 increases from 61.6% to 86.6% using a GPT-2 architecture trained with synthetic arithmetic data. At all values of $k \in \{1, 4, 8\}$, DR-PSR shows performance beyond standard PSR.

The important points we discovered were:

1. the trend observed with PSR about diversity reduction has been measured through entropy and we do find the same steep reduction even with a 50-task dataset trained using GPT-2,

2. the reduction in entropy during training seems to follow output diversity for each k which is clearly observable across runs. The recovery of diversity follows directly from maintained entropy, where improved performance at high k is seen,
3. the best range for accuracy-diversity trade-off is between $\beta = 0.02-0.05$. DR-PSR has good stability in this range, although optimal values may vary according to each specific configuration.
4. No negative examples were required which allows the approach to be used in those situations where negative sample generation is impossible or where it becomes unreliable.

In comparison with W-REINFORCE our approach provides the same output without relying on negative examples and using a much lighter framework, meaning full RLVR behaviour can be achieved solely through training with positive samples. Future steps in development include scaling this approach to larger architectures and testing across more challenging benchmarks.

Acknowledgements

This work is an extension of Zhu et al. (NeurIPS 2025) [1]. We thank the original authors for their clear decomposition framework and for open-sourcing their code at <https://github.com/TianHongZXY/RLVR-Decomposed>.

References

- [1] Xinyu Zhu, Mengzhou Xia, Zhepei Wei, Wei-Lin Chen, Danqi Chen, and Yu Meng. The Surprising Effectiveness of Negative Reinforcement in LLM Reasoning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2506.01347.
- [2] Daya Guo, Dejian Yang, Haowei Zhang, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv:2501.12948, 2025.
- [3] Zhihong Shao, Peiyi Wang, Qihao Zhu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv:2402.03300, 2024.
- [4] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization algorithms. arXiv:1707.06347, 2017.
- [5] Kimi Team. Kimi K1.5: Scaling reinforcement learning with LLMs. arXiv:2501.12599, 2025.
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on code. arXiv:2107.03374, 2021.
- [7] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019.
- [8] Xingyu Dang, Christina Baek, Kaiyue Wen, Zico Kolter, and Aditi Raghunathan. Weight ensembling improves reasoning in language models. arXiv:2504.10478, 2025.
- [9] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *ICML*, 2018.
- [10] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, et al. Training verifiers to solve math word problems. arXiv:2110.14168, 2021.